Scope: which name does Python mean?

Local names live in a call's frame and vanish with it; Python looks a name up in four places, in a fixed order.

THE QUESTION

When a function and the rest of the program both use a name `y`, which `y` does a line mean?

THE INTUITION

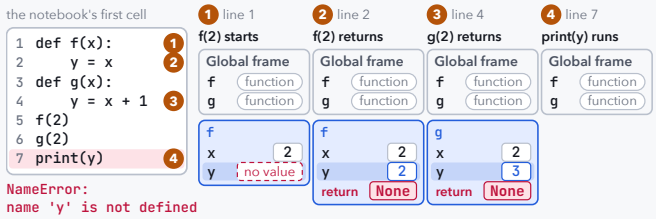
Each call gets a fresh frame for its own names. A name assigned in a function belongs to that frame for the whole function. Other names are looked up outward: the def around it, the file, Python's built-ins.

KEY TAKEAWAY

Get values in through parameters and out with `return`. Reading an outer name is fine; changing one needs `global`, which you rarely want.

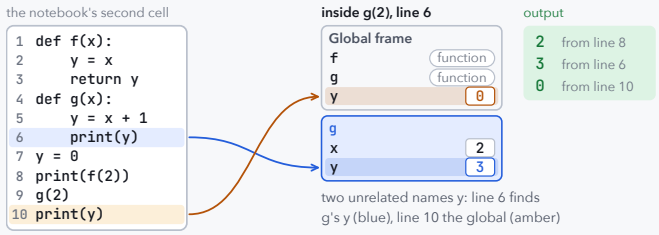
a name → L: the running call → E: the def around it → G: the file → B: built-ins → none: NameError

1 Names made in a call exist only while it runs



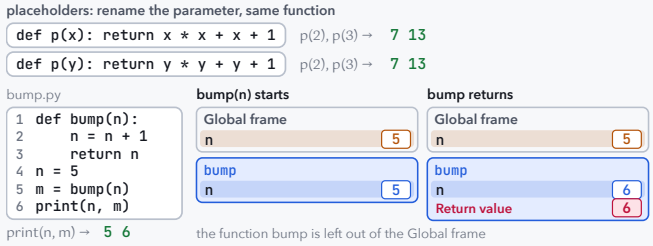
The **scope** of a name is the part of a program where it can be used. A name assigned in a function is a **local variable** in that call's **frame** (the functions sheet; blue while it runs), made at the call and removed at the return; `y` is listed from the start, without a value (panel 5). At line 7 only the **Global frame**, the file's **global variables**, is left.

2 A local hides a global of the same name



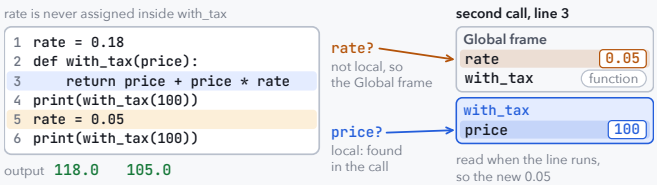
Now the file has its own `y = 0`. Inside `g`, `y = x + 1` makes a second, local `y` that hides the global one while `g` runs: **shadowing**. Line 6 prints 3 and line 10 prints 0: the global `y` never changed.

3 Parameters are local names too



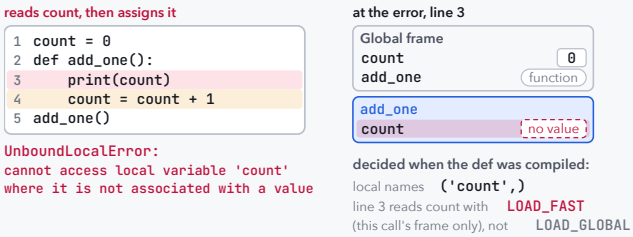
A **parameter** is a local variable that the call fills with the argument. Its name is a **placeholder**, like x in $p(x) = x^2 + x + 1$. `n = n + 1` moves only `bump`'s `n`; the caller's `n` stays 5. To get a new value out, `return` it.

4 A function can read a global name



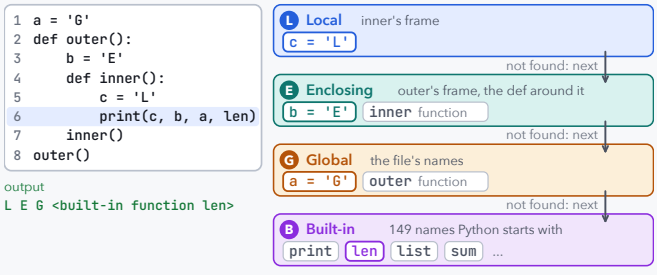
Python looks for a name in the running call's frame first. `rate` is not there, so it comes from the Global frame, looked up each time the line runs: after line 5 the same call gives 105.0. Reading a global needs nothing special; assigning to one does (page 2).

5 Assigned anywhere in a def, local everywhere in it



Python **compiles** a whole `def` before running any of it, and fixes its local names then: a name assigned anywhere in it (`=`, `+=`, a `for` variable) is local throughout, even above the assignment, even under `if False:`. Line 3 looks only in the call's frame and finds no value: **UnboundLocalError**. Fix: pass `count` in and `return` the new value.

6 LEGB: the order Python searches for a name



Python searches four **scopes** and stops at the first match: **Local**, the running call; **Enclosing**, the call of a `def` around it (panel 10); **Global**, the file; **Built-in**, names Python starts with. Found nowhere, as with `print(total)` on line 6: `NameError: name 'total' is not defined`.

Changing names outside a function, and when not to

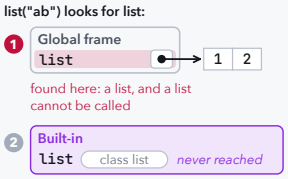
Hiding a built-in by accident, `global` and a rewrite without it, lists changed in place, and a first look at `nonlocal`.

7 Call a variable `list` and the built-in is hidden

one REPL session

```
>>> list("ab")
['a', 'b']
>>> list = [1, 2]
>>> list("ab")
TypeError: 'list' object
  is not callable
>>> del list
>>> list("ab")
['a', 'b']
```

list("ab") looks for list:

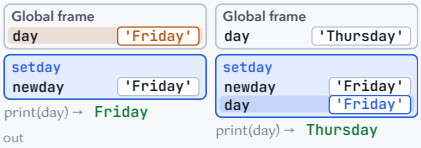


`list = [1, 2]` puts `list` in the Global frame, so the search stops before the Built-in level and `list("ab")` calls a list: **TypeError**. Naming a variable `sum`, `max`, `str` or `input` breaks those the same way. `del list` brings the built-in back; better, choose names such as `nums`.

8 `global` rebinds a global name; avoid it

the notebook's setday

```
1 day = "Thursday"
2 def setday(newday):
3     global day
4     day = newday
5 setday("Friday")
6 print(day)
```



frames at setday's return; functions left out

before: a global counter

```
def add_points(p):
    global score
    score = score + p
```

two calls, then print(score) → 8

after: pass it in, return it

```
def add_points(score, p):
    return score + p
score = add_points(score, 5)
```

also 8; add_points(10, 5) → 15

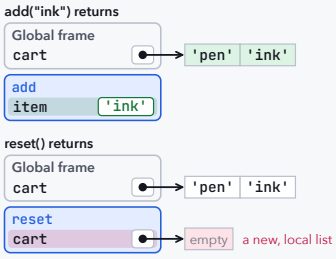
`global day` says `day` in this function is the Global frame's `day`, so line 4 changes it; without it, line 4 makes a local. Avoid `global`: a call changes something you cannot see at the call, and the function works on that one variable only. Pass values in, `return` results.

9 A global list can be changed in place

```
1 cart = []
2 def add(item):
3     cart.append(item)
4 def reset():
5     cart = []
6 add("pen")
7 add("ink")
8 reset()
9 print(cart)
```

prints `['pen', 'ink']`

functions are left out of the Global frames



`cart.append(item)` assigns nothing: it reads `cart`, finds the global list and changes that object (the names and objects sheet). `cart = []` is an assignment, so `reset` gets a local `cart` and the global keeps its items; `cart.clear()` would empty it. `cart += [item]` assigns too: UnboundLocalError.

10 The enclosing level: a def inside a def

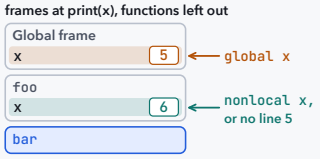
the notebook's foo and bar

```
1 x = 5
2 def foo():
3     x = 6
4     def bar():
5         nonlocal x
6         print(x)
7     bar()
8 foo()
```

line 5 as written, swapped, deleted:

`nonlocal x` → 6 | `global x` → 5 | (no line 5) → 6

frames at print(x), functions left out



nonlocal outside a nested def:

```
SyntaxError: no binding
for nonlocal 'day' found
```

`bar` has no `x`, so the search moves to E, `foo`'s frame: 6, not the global 5. `global x` jumps to the file's `x`; `nonlocal x` means the nearest enclosing function's `x` and lets `bar` assign to it. Inner functions that outlive the outer call: the closures sheet.

11 Common mistakes: what you wrote, what Python says, the fix

you wrote	the last line Python prints; why, and the fix
`def area(r): a = 3.14 * r * r` `area(2); print(a)`	`NameError: name 'a' is not defined` a was local to `area`. `return` it and keep the result: `a = area(2)` gives 12.56
`def add(n): total += n` `total = 0; add(5)`	`UnboundLocalError: cannot access local variable 'total' where it is not associated with a value` `+=` assigns, so `total` is local and unset. Pass it in and return the sum
`sum = 0; sum += 1` `print(sum([3, 4]))`	`TypeError: 'int' object is not callable` the global `sum` hides the built-in. Call your variable `total`: 7
`def f(): x = 1; global x`	`SyntaxError: name 'x' is assigned to before global declaration` put `global x` first, before any use of `x` in the function; better, avoid `global`

12 Try it: predict the output (answers in green)

1. `x = 4; f(x)` returns `x * 2`. `print(f(3), x)`? → `6 4`: the parameter is local.
2. `n = 1; g()` sets `n = 5`, returns it. `print(g(), n)`? → `5 1`: g's `n` is local.
3. `t = 10; h()` returns `t + 1`; `t = 20`. `h()`? → `21`: read when `h` runs.
4. `v = 1; k()` prints `v`, then sets `v = 3`. `k()`? → UnboundLocalError: `v` is local.
5. `xs = [1]`; `m()` does `xs.append(2)`. Call `m()` twice; `xs`? → `[1, 2, 2]`: changed in place.
6. `x = 'g'`; `outer()` sets `x = 'e'`, returns `inner()`, which returns `x`. `outer()`? → `'e'`: E before G.

a call's names live in its frame · assigning makes a name local · search L, E, G, B · pass in, return out

Source: the scope notebook and its handwritten lecture notes, nipunbatra.github.io/comp22; `bump`, `with_tax`, `add_points`, `cart` and the practice problems are our own. Outputs come from running the code with Python 3.11.11; every frame is a `sys.settrace` snapshot of the running code, and the level of each name is read off the compiled function. Draw your own at pythontutor.com. Next: the closures sheet.